AI Engineering and Claude Architect · Part III, Agents

AI Frameworks, Lesson 1

Do You Need a Framework? SDK to Managed Agents

A team adopted a framework to save a week. Six months later a new model rejected a parameter that the framework sent by default, the fix wasn't released yet, and nobody on the team could say what the framework was actually sending.

Module	AI07 of 13
Lesson	1 of 4
Prerequisites	AI03 L1, AI06 L1–L3
Exam relevance	Third-party frameworks are out of scope. The Agent SDK appears in Scenario 4, supporting Domains 1 and 2
Reading time	~30 minutes

Contents

1. Do You Need a Framework?

Lesson 1 of 4 · AI Frameworks

Do You Need a Framework?

Key idea

A framework trades code you'd have to write for behavior you can't see. That's often a good trade, as long as you know what you've given up. **Choose by who owns the harness and who owns the deployment, keep the framework behind a small interface of your own, and read what it sends before you rely on it.**

> *Before: the agentic loop (AI06 L2). You should be able to write one before you hide one. Teaches: the options from raw SDK to hosted agents, what frameworks hide, how to stay able to leave. Used by: AI07 L2–L4, AI08.*

The hook: what a framework decides for you

Here's a measurement. Later lessons run real LangChain4j, Spring AI and LangGraph code against scripted models, and they found decisions most developers never see:

Library	What it did, unasked	Lesson
LangChain4j 1.23.0, default settings	Sent a raw exception message to the model, including an internal IP address	L4
Spring AI 2.0.2, default settings	Same thing	L4
LangChain4j	Stopped a runaway loop at 100 round trips, with an exception	L4
Spring AI	Stopped at 40 calls to one tool, and **returned the limit message as the answer**	L4
Spring AI	Turned a 17-character request into 667 characters, to ask for JSON	L4
LlamaIndex 0.14.25	Wrote the whole RAG prompt	L3
LangGraph 1.2.20	Restarted a paused node from its first line on resume	L2

None of these is wrong. Each is a reasonable default. But each one is a production decision that somebody made for you, so you ought to know about it.

Intuition: a meal kit or a restaurant

Option	Cooking analogy	You control	You give up
Raw SDK	Buy ingredients	Everything	Time
Framework	A meal kit	Some choices	Seeing what's in the sauce
Hosted agent	A restaurant	The order	The kitchen

A meal kit is great on a weeknight. If you have an allergy, though, you read the label. In production, you always have allergies.

Mechanism: two questions

The Claude API reference separates these neatly: **who supplies the harness** (the loop and context management), and **who supplies the deployment** (where it runs).

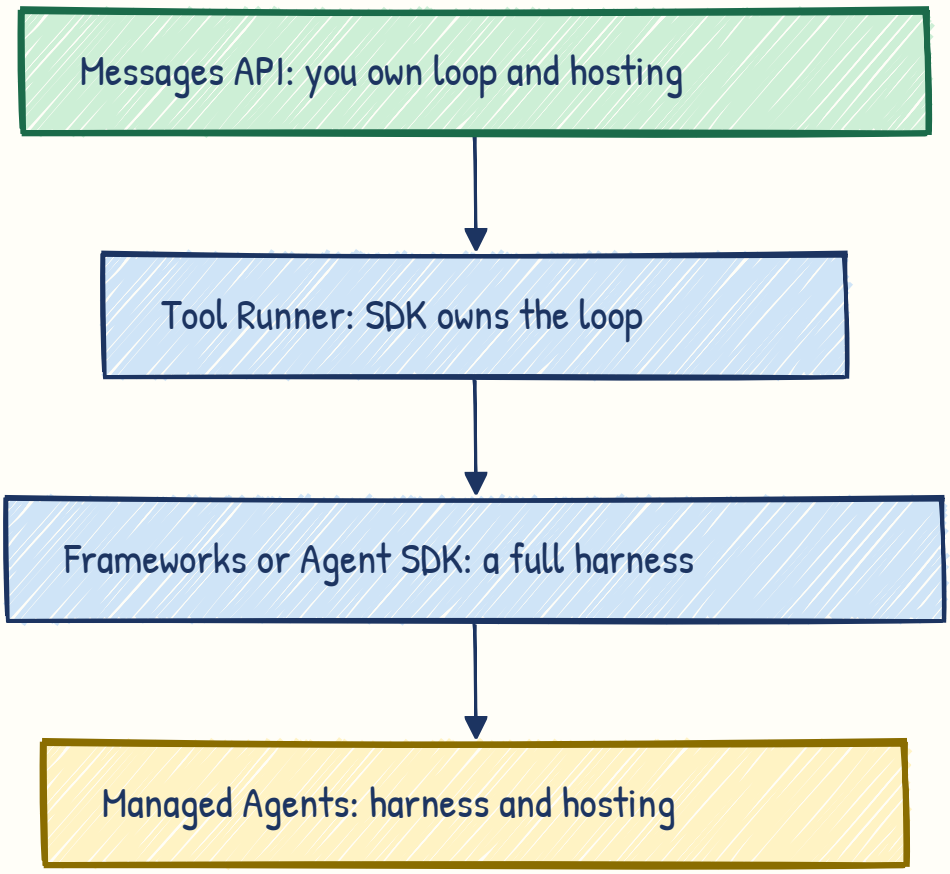


Figure 1. Options by what you still own. Moving down, you write less, and you see and control less.

Option	Harness	Deployment	Tools you get	Languages
Messages API	You	You	Only yours	All SDKs
Tool Runner (beta)	SDK	You	Only yours	All SDKs
Agent SDK	Claude Code's	You	Files, bash, search, plus MCP	Python, TypeScript
Third-party framework	Theirs	You	Integrations	Varies
Managed Agents (beta)	Anthropic's	Anthropic's, or your own sandbox	Sandbox tools, Skills, MCP	SDKs, REST, `ant` CLI

A note for Java teams: the Agent SDK is Python and TypeScript. To drive the same loop from Java, its docs say you can run the CLI as a subprocess with `-p` and `--output-format json`. Managed Agents has a Java SDK surface. Otherwise, use the Messages API or a Java framework.

What each one adds

You want	Reach for
Single call or fixed workflow	Messages API
Custom-tool agent	Tool Runner
A coding or filesystem agent with hooks and subagents	Agent SDK
Persisted, versioned agent configs, hosted long-running sessions, schedules	Managed Agents
Many model providers behind one interface	A framework
Spring or Quarkus integration, with vector stores	Spring AI or LangChain4j
Durable graphs with human approval, and you're on Python	LangGraph
Heavy document ingestion, and you're on Python	LlamaIndex

Managed Agents in brief

Per the Claude API reference: you create a persisted, versioned **Agent** once, then start a **Session** per run, each with its own container. The loop runs on Anthropic's side, and the tools run in the container, or in a self-hosted sandbox.

Feature	Detail
Versioning	Each change makes a new immutable version, and sessions pin to one
Credentials	Vaults, substituted at egress and never visible in the sandbox
Budgets	Hard dollar caps per session
Outcomes	A grader iterates against your rubric
Status	**Beta**

The trade-off: files must be uploaded or mounted, and it isn't on every cloud platform.

What frameworks actually give you

Value	Real, when…
Provider portability	You really will switch, and you aren't relying on provider features such as caching or citations
Integrations	You need many vector stores and loaders
Plumbing	Tool schemas from annotations, memory, streaming
Patterns	Durable execution, human approval
Ecosystem	Tracing tools and hiring

What they cost

Cost	Why it matters
Hidden prompts	Anthropic's post warns that abstractions "can obscure the underlying prompts"
Lagging features	New API features wait for support. Spring AI's 2.0.2 page still shows Claude 4 model names, and `temperature` options that current models reject
Lowest common denominator	Portable interfaces can flatten provider-specific features
Upgrade churn	Fast-moving APIs. Spring AI 2.0 deprecated `getDefaultOptions()`, for instance, which cost me a debugging session in this module
Dependencies	See the table below
Debugging	Stack traces through layers you didn't write

A real dependency count

Measured with Maven in throwaway projects, on 5 November 2026:

Option	Jars	Size
`anthropic-java` 3.2.0	29	49.0 MB
LangChain4j 1.23.0 core and Anthropic	13	5.5 MB
Spring AI 2.0.2 client and Anthropic	56	65.9 MB

This surprised me. The official SDK is heavier than LangChain4j, because `langchain4j-anthropic` has its own HTTP client and doesn't depend on `com.anthropic` at all. Spring AI does use the official SDK, but pins `anthropic-java-core` 2.?2.0, an older major version than the 3.2.0 this course uses.

So a framework's model support doesn't simply track the SDK. **Size alone shouldn't decide this**, but it shows why you should look, not assume.

Why the obvious fixes fall short

Tempting move	Why it disappoints	Better
"Frameworks are bloat, so write everything"	You'll rebuild memory, schemas and retries, with more bugs	Use one where it's plainly earning its keep
"Everyone uses X, so use X"	Popularity doesn't fit your constraints	Decide by harness, hosting and language
Choose for portability, then use caching everywhere	You're provider-locked anyway	Be honest about which you need
Spread framework types through business code	Leaving becomes a rewrite	Wrap it behind your own port
Trust the defaults	They're production decisions	Log and read actual requests

How Anthropic approaches it

From Building Effective Agents:

- → Frameworks simplify calls, tools and chaining, while they can hide prompts and complicate debugging.
- → So begin with LLM APIs directly, and understand any framework's underlying code, since wrong assumptions cause many customer errors.
- → When moving to production, reduce abstraction layers, and build with basic components.

Note that Anthropic ships its own harnesses as well: the Agent SDK, Tool Runner and Managed Agents. So the advice isn't "never use a harness". It's "know what's in it".

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: the framework everywhere. Controllers, services and tests all use framework types.

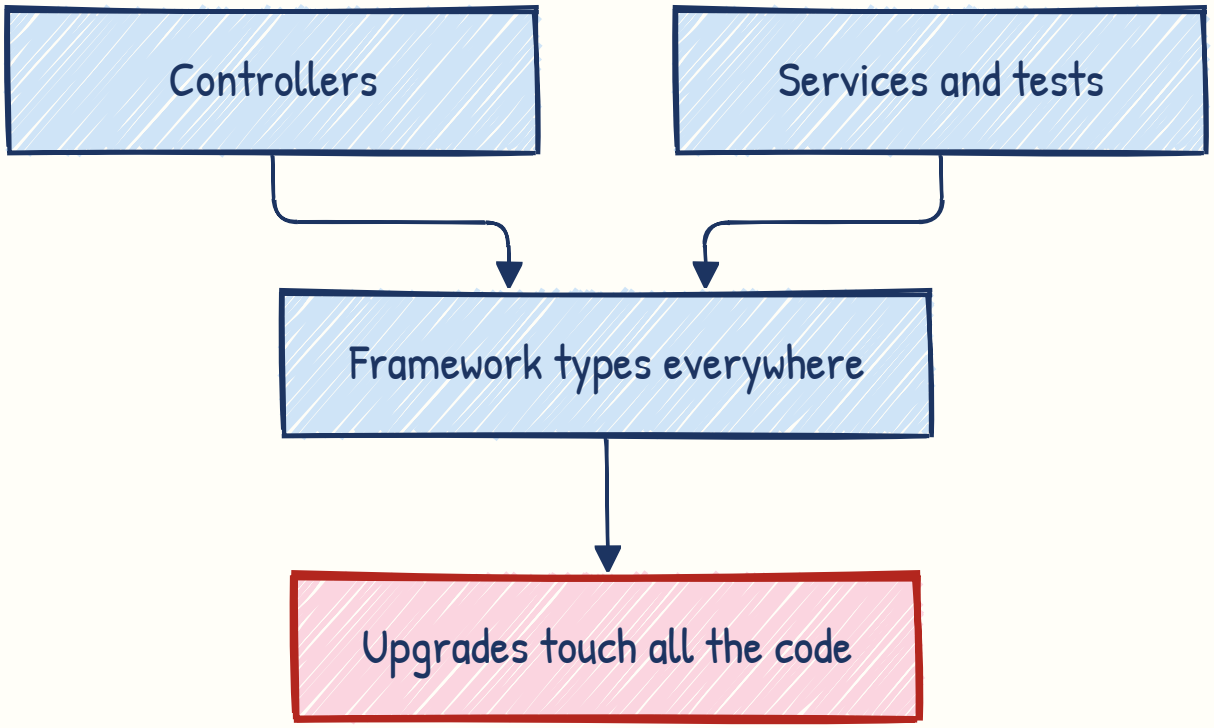


Figure 2. Baseline. Framework types spread through the code, so an upgrade or a change of library touches everything.

Improved: raw SDK only. It's clear, and nothing's hidden. New problem: **you're now maintaining your own memory, tool schemas and retries, and each team writes its own.**

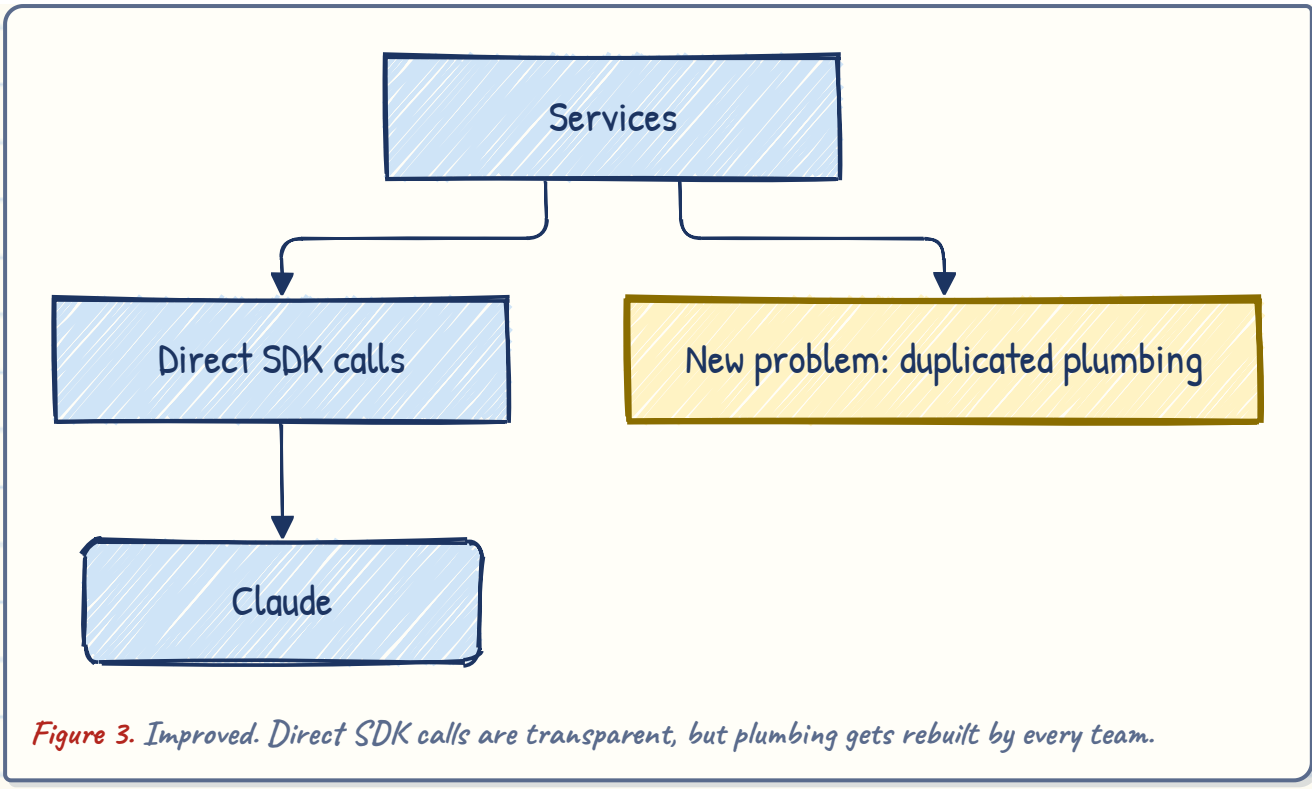


Figure 3. Improved. Direct SDK calls are transparent, but plumbing gets rebuilt by every team.

Production: a port, with one adapter.

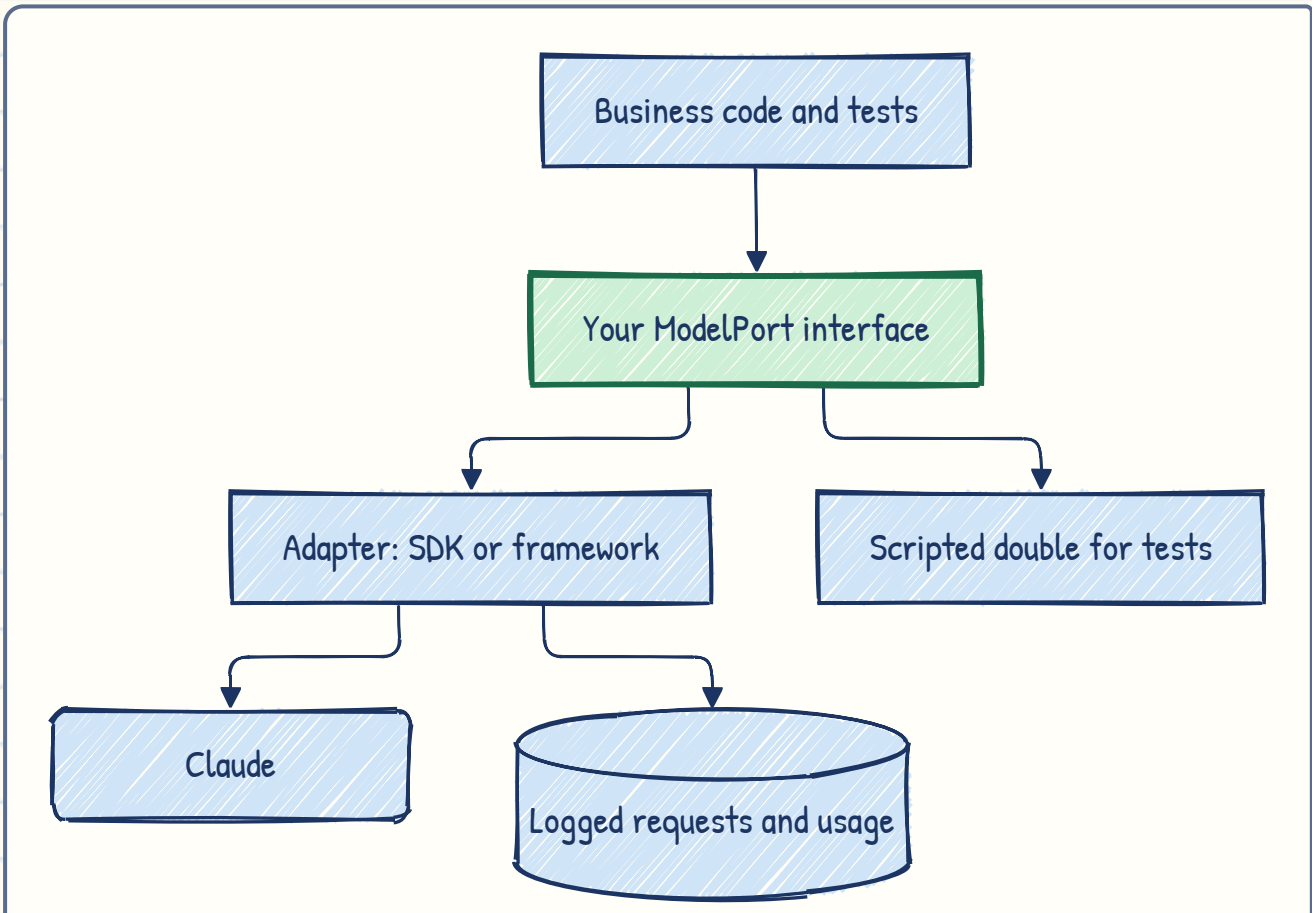


Figure 4. Production. Business code depends on your own small interface. One adapter holds the chosen library, and every real request is logged for inspection.

Component	Why it exists	How it communicates
`ModelPort`	Business rules don't know the library	Your own types
Adapter	The only code that imports the framework	Calls the library
Request log	Shows what's really sent	Async
Scripted double	Fast tests with no key	In-process
Upgrade test	Catches changed defaults	CI runs against a real model, nightly

Failures

Failure	Containment
New model rejects a framework default	Nightly test on your actual model, and an override through the adapter
Framework upgrade changes behavior	Pin versions, and diff logged requests
Feature missing from the framework	Drop to the SDK inside that adapter
Framework abandoned	Replace one adapter. AI07 L3 shows this really happens

Java

From `ModelPort.java`. Business rules depend only on this, so they run with no key, no network and no framework. Real output:

```
public interface ModelPort {
```

```
Reply send(Request request);
```

```
t1 -> Customer was charged twice.  spent=130
t2 -> TRUNCATED                    spent=480
t3 -> FAILED: refusal              spent=570
t4 -> BUDGET_EXCEEDED              spent=570
Model requests made: 3
```

The rules here are yours, so an upgrade can't quietly change them: truncation isn't treated as success, a refusal isn't swallowed, and the budget stops request 4.

Trade-offs

Decision	Gain	Cost
Raw SDK	Transparency, newest features	More plumbing
Framework	Speed, integrations	Hidden behavior, lag, churn
Agent SDK	A proven harness	Python and TypeScript, and it's built around Claude Code's tools
Managed Agents	No loop or sandbox to run	Beta, less control, platform limits
A port	Freedom to change	A thin layer to maintain, and it can hide features if badly designed

When a port is overkill: a prototype, or a small script.

Common mistakes

1. **Adopting a framework before writing a loop.** You can't debug what you don't understand.
2. **Assuming portability is free.** Provider features aren't portable.
3. **Never reading the real requests.** Turn on logging once, and read them.
4. **Trusting version numbers as compatibility.** Test on your actual model.
5. **Leaking framework types into business code.** Keep them in an adapter.
6. **Choosing on popularity.** Choose on harness, hosting and language.

What to say in an interview

"I'd decide by who owns the harness and the deployment. For one custom agent, the Tool Runner may be enough. In a Spring shop needing vector stores, Spring AI earns its place. Either way, I'd wrap it behind a small port, log real requests, and run a nightly test on our actual model, because framework defaults are production decisions, and new models can reject them."

Certification lens

How this lesson maps to CCAR-F

LangChain, LlamaIndex and similar frameworks aren't on the exam. The Agent SDK is, in Scenario 4. Know that it provides built-in tools, hooks, subagents and sessions, and that it isn't the same as the Tool Runner.

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Component selection · Medium. Pick an option, and give a reason: (a) a Python team wants a coding agent with file tools and hooks, running on their servers; (b) a Java team has one custom-tool agent; (c) a team wants scheduled agents, with no infrastructure to run; (d) a Spring team needs chat with three vector stores.

A (a) Agent SDK. (b) Tool Runner, or a manual loop. (c) Managed Agents, noting that it's in beta. (d) Spring AI, for its integration and vector store support.

Q Q2 · Troubleshooting · Medium. After moving to a new Claude model, every framework call returns a 400. The framework hasn't changed. What do you check first, and how?

A What the framework sends by default. Log the raw request, and compare it with the model's migration notes. Likely causes are parameters that the model now rejects, such as sampling settings or forced tool choice. Override them in configuration, or drop to the SDK in the adapter, and add a nightly test on the model you actually use.

Q Q3 · Best decision · Hard. A colleague wants a framework "so we can switch providers later". The design relies on prompt caching, citations and strict tool use. What do you say?

A Those are provider features, so switching wouldn't be free anyway, and a portable layer might hide them. Portability has value only if switching is a real plan. If it is, put the provider-specific code behind a port with an adapter per provider, and test each one on evals. If it isn't, choose on other grounds.

Revision - Lesson 1

→ A framework trades code you'd write for behavior you can't see.

→ Ask who owns the harness and who owns the deployment.

→ Messages API, Tool Runner, Agent SDK or frameworks, Managed Agents.

→ The Agent SDK is Python and TypeScript. From Java, call the CLI or use the Messages API.

→ Tool Runner and the Agent SDK aren't the same thing.

→ Framework defaults are production decisions, so read real requests.

→ LangChain4j uses its own client. Spring AI pins an older major of the official SDK.

→ Anthropic: start with the API, and know what any framework does.

→ Keep frameworks behind your own port, with one adapter.

→ Run a nightly test on your actual model.

Your notes